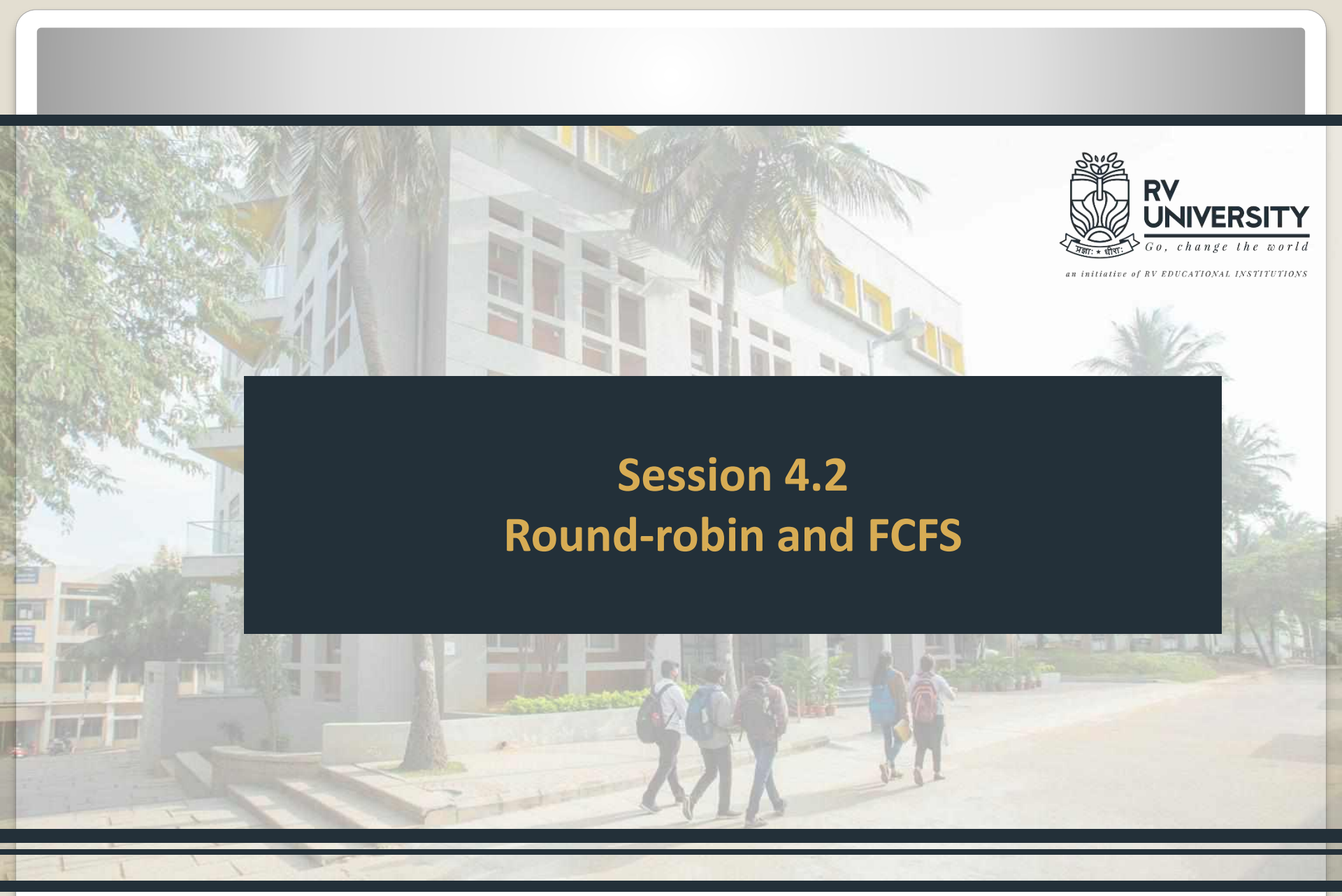


RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

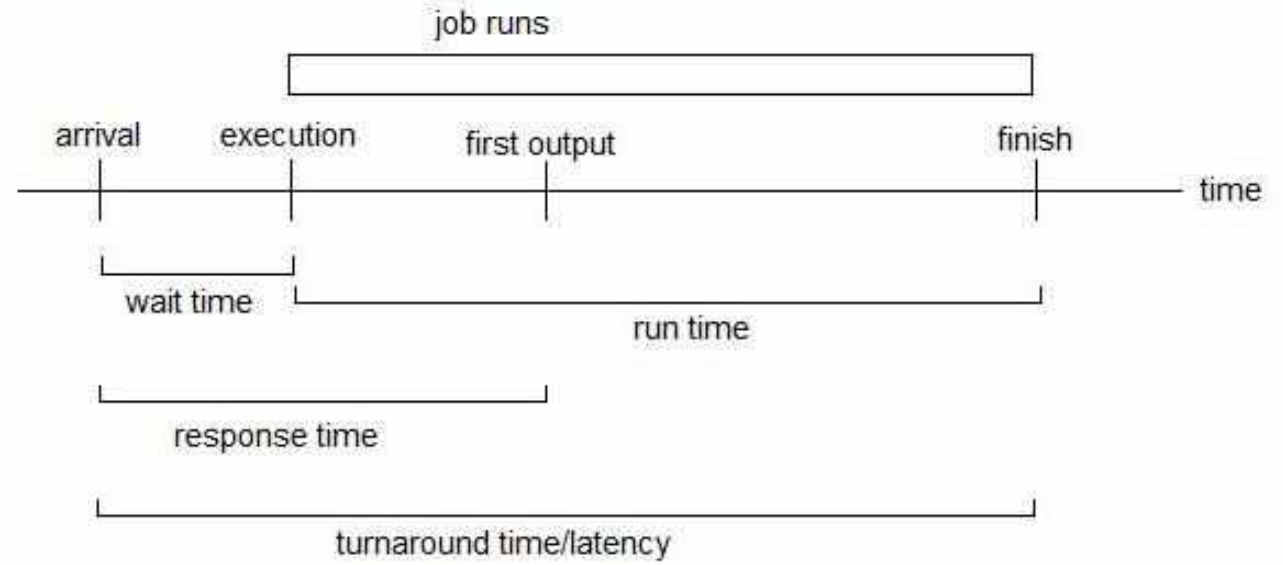
Session 4.2
Round-robin and FCFS

Session 4.2: Focus

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- Scheduling Criteria - Recap
- Quiz 1: Waiting time
- Round-robin Scheduler
  - Examples 1 and 2
- FCFS Scheduler
- Round-robin: Example 3
  - Time-line and Ready Queue
  - Homework: Scheduler Criteria computation

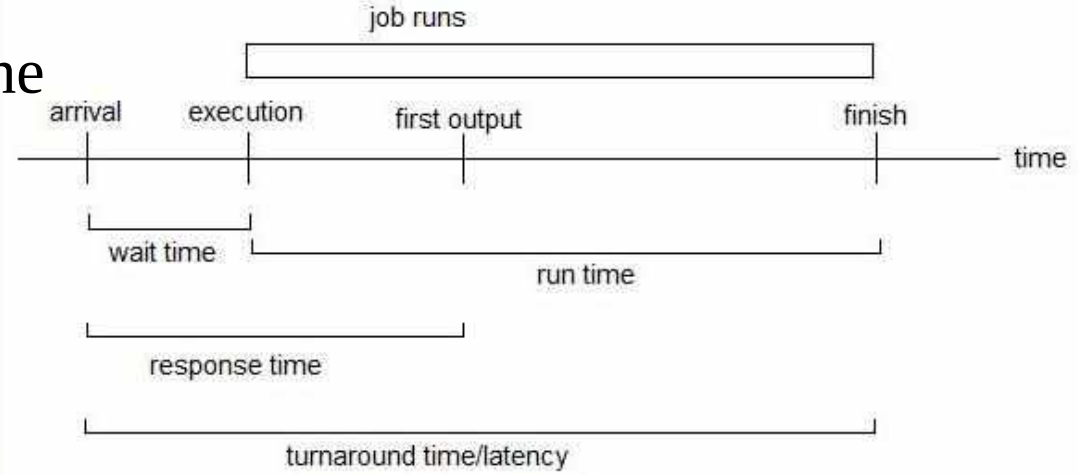
Scheduler Performance Related Parameters -Recap



- Scheduler performance is measured in terms of various timing parameters associated with processes or jobs being scheduled by the Scheduler, towards their completion at the earliest by utilizing the CPU efficiently.

Scheduling Criteria - Recap

- **CPU utilization** – Keep the CPU as busy as possible productively.
- **Throughput** – No. of processes that complete their execution per time unit.



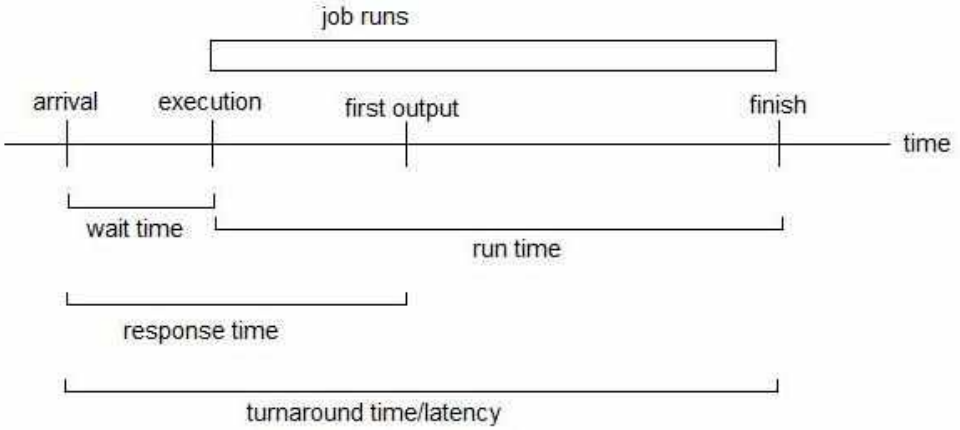
- **Turnaround time** – Total amount of time taken to execute and complete a particular process, from the time the process was ready.
- **Waiting time** – Total amount of time a process has been waiting in the ready queue.
- **Response time** – Amount of time taken, from the time when a job request was submitted (job released) till the first response it produced (for interactive jobs).

Quiz 1: Choose all the Correct Options

- **Waiting time** – **Total** amount of **time** a **process** has been **waiting** in the **ready queue**

Note: Remember that we are analyzing how good a scheduler is, using this criterion.

Choose all the correct options.



Q1: Why is the **waiting time** of a **process** in the "**blocked queue**" not considered here?

A. Ready queue is only connected directly to the CPU.
B. Blocked queue waiting is not because of the scheduler.
C. A process wants to do an I/O and gets blocked for its own requirements, the scheduler of the OS has no role to play.
D. A process will be in the Ready when it has all the resources except the CPU, due to the scheduler not allowing it to run on the CPU.

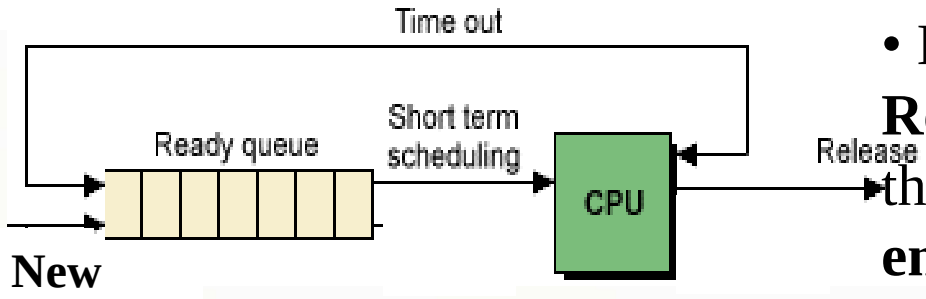
Note: B is also the right choice, because it is a correct Statement.

ANS: B, C and D **What about the time spent in "Ready, suspend queue"?**

Answer: "Ready, suspend Queue", if present, the time spent there should also be considered, because scheduler decides to move a process to this queue, which is already in Ready queue.

Round-Robin Scheduler

Round-Robin (RR) Scheduler



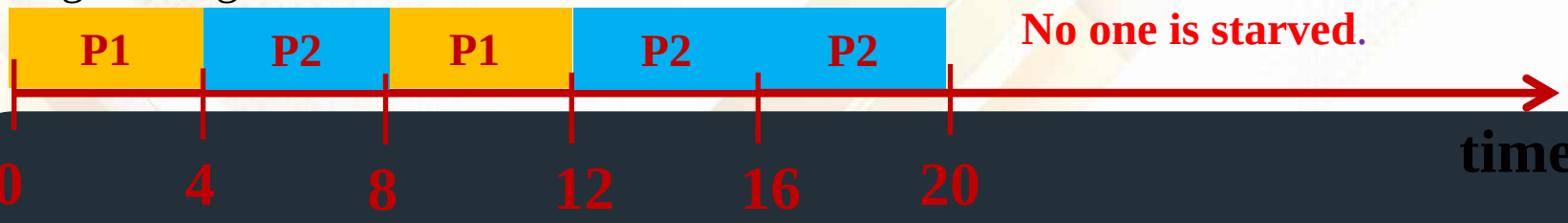
• It **picks** a **process** at the **head** of the **Ready Q** to be **scheduled** and moves the **currently running process** to the **end** of the **Ready Q**, if it is in Ready state.

- **RR** is appropriate for time-sharing environments.
- **Quantum Time** (**q**): Amount of time a process gets before being context switched out (also called **time slice**),
  - It is also configurable.
  - A suitable value is chosen based on the type of applications running in the system and the kind of response needed to be provided by the OS.
- **Context switching time** value is **significant** though it is expected to be less than **1% of Quantum time.**
- If **n processes** are running under **round-robin** they will all have an illusion that they have **exclusive access to a CPU** which is running at **1/n times** the **actual processor speed**.

Example 1: RR

- Suppose both P1 and P2 processes arrive at time **zero**
- P1: C1 = 8 and P2: C2 = 12 (computation time for each)
- Assume **preemptive** scheduling with **Quantum = 4 time units**
- Consider the time taken for the dispatch of each process as negligible
- Compute the following:
  - Throughput: **= 2/20 = 0.1**
  - CPU utilization: **= 100%**
  - Avg. Turnaround: **= (12 + 20)/2 = 16**
  - Avg. Wait time: **= (4 + 8)/2 = 6**

Non-preemptive case:
Avg Turnaround time = **14**
Avg Waiting time = **4**



Note 1: When two processes arrive at the same time, process with a lower ID is added to the queue first.

Note 2: Compared to non-preemptive scheduling, both **Avg Turnaround** and **Avg Wait time** have increased here. But **preemptive RR scheduler** helps **all** the **Ready processes** in the system to make **progress**, giving **equal time share** of the **CPU** to **all**.
No one is starved.

Example 2: RR

- Suppose both P1 and P2 processes arrive at time **zero**
- P1: C1 = 8 and P2: C2 = 12 (computation time for each)
- Assume **preemptive** scheduling with **Quantum = 3 time units**
- Consider the time taken for the dispatch of each process as negligible
- Compute the following:
  - Throughput: **= 2/20 = 0.1**
  - CPU utilization: **= 100%**
  - Avg. Turnaround: **= (14 + 20)/2 = 17**
  - Avg. Wait time: **= (6 + 8)/2 = 7**

Note: When a process completes, the scheduler kicks in and schedules the other process in the Ready Queue, if there is one, even if the quantum time is not over.

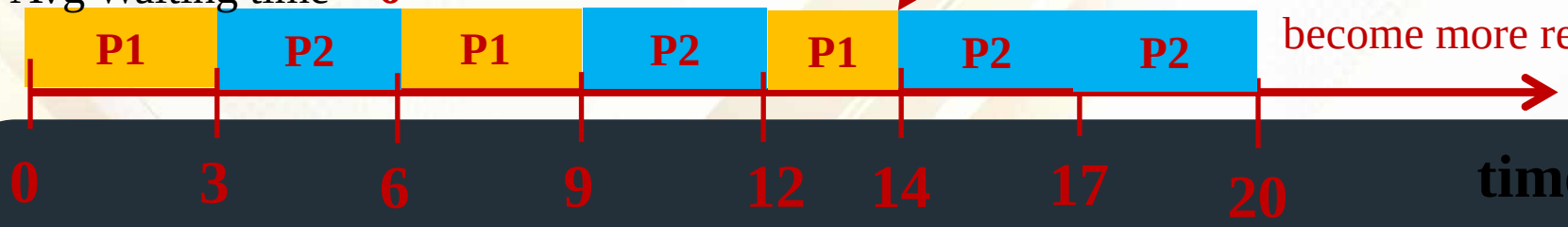
What is the **impact** of **reducing** the **time slice** from **4 to 3 time units**?

ANS: Since the scheduler is made to run more often here, turnaround and wait time increases, but the tasks become more responsive.

With Quantum time 4 time units: (**Example 1**)

Avg Turnaround time = **16**

Avg Waiting time = **6**



FCFS Scheduler

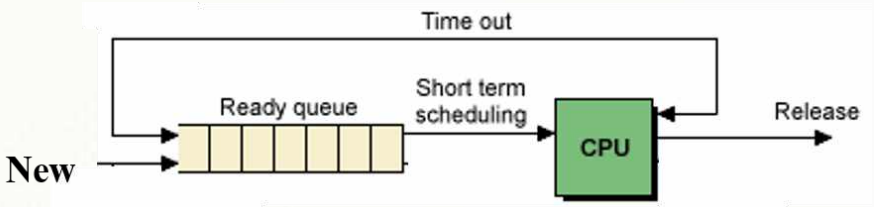
- **FCFS**: First-Come-First-Served or FIFO (First-In-First-Out).
- This is a **Round-Robin scheduler** with **Quantum = ∞.**
- The processes are allowed to run until they either complete their execution or they get blocked because of making an I/O call or blocking system calls.
- It is effectively a **non-preemptive scheduler.**
- Processes can get starved if the processes which have arrived earlier, take long time to complete.
- This is not a fair scheduler.
- It favours the processes which are in the system earlier than other processes.

Example 3: Draw the time-line of all the Processes in the Table below

In 10 minutes

RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Burst time** is same as **computation time** or **execution time.**
- Assume **RR** scheduler with **time slice = 2 time units.**
- Only draw the **order** in which various **processes get executed** until their **completion**. **Show** the **Ready Q status** too, whenever scheduler gets called.
- Computing scheduler criteria is **Homework**



Job	Arrival Time	Burst Time
A	0	6
B	1	2
C	2	5
D	3	2
E	7	1

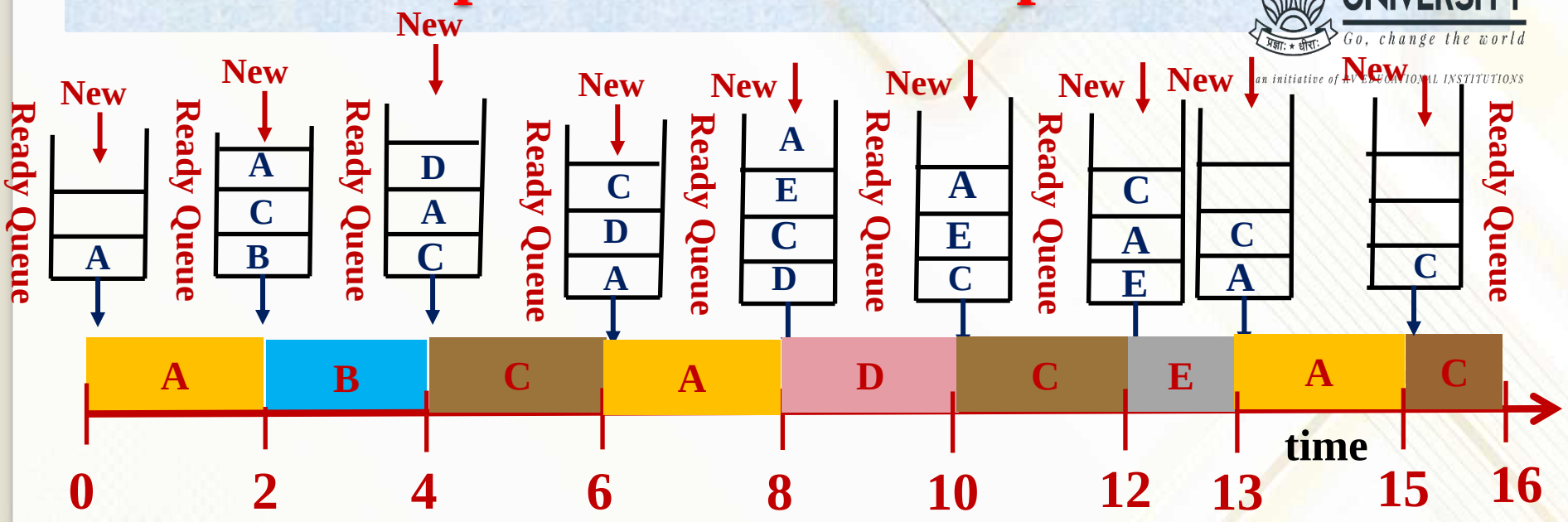
Note: Assume at a time instant a newly arrived Process enters the Ready queue earlier than the Process evicted out of the CPU at the same time instant.

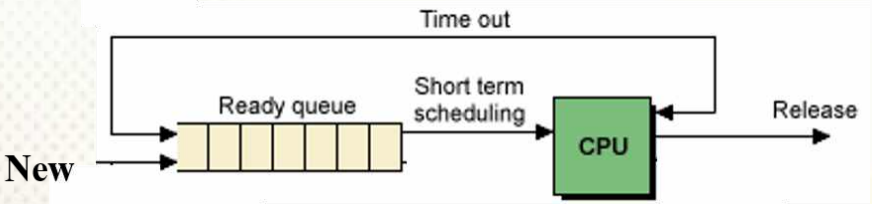
For example, at time 2, Job C will get added to the Ready Q before the process which is currently running on CPU is moved into it.

You can do it in groups too!!!

Chocolate for the First correct answer!!!

Example 3: Solution - Updated





Job	Arrival Time	Burst Time
A	0	6
B	1	2
C	2	5
D	3	2
E	7	1

Session 4.2: Summary

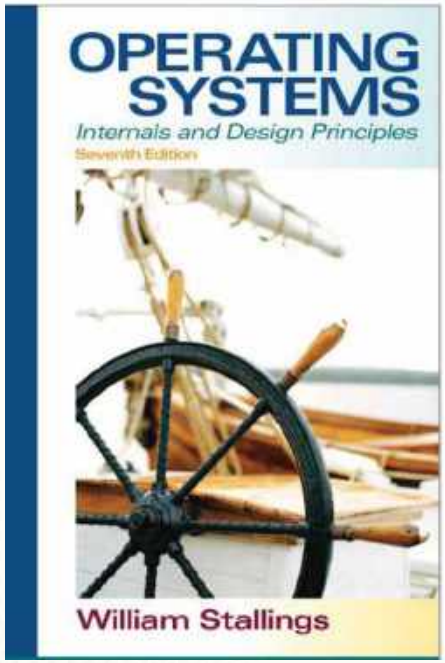
RV UNIVERSITY
Go, change the world
an initiative of RV EDUCATIONAL INSTITUTIONS

- **Scheduling Criteria - Recap**
- **Quiz 1: Waiting time**
- **Round-robin Scheduler**
  - **Examples 1 and 2**
- **FCFS Scheduler**
- **Round-robin: Example 3**
  - **Time-line and Ready Queue**
  - **Homework: Scheduler Criteria computation**

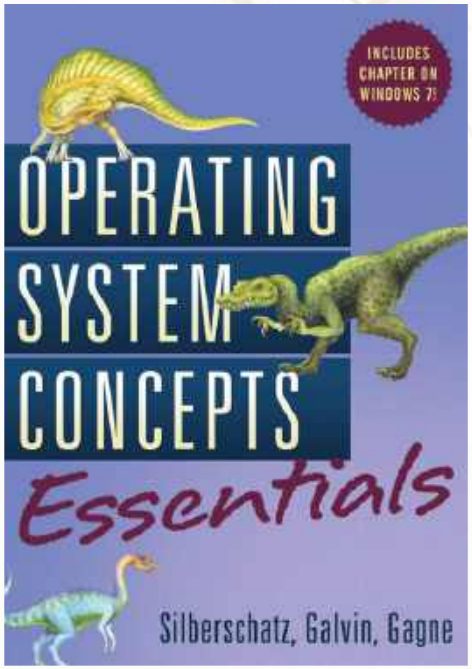
References (1 to 5)

RV UNIVERSITY
Go change the world

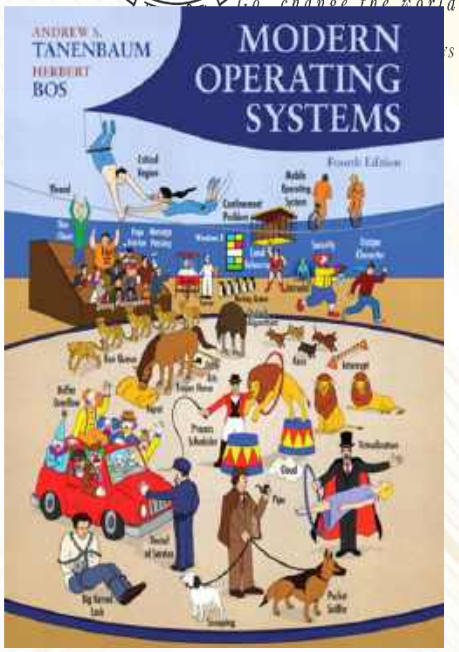
Ref 1



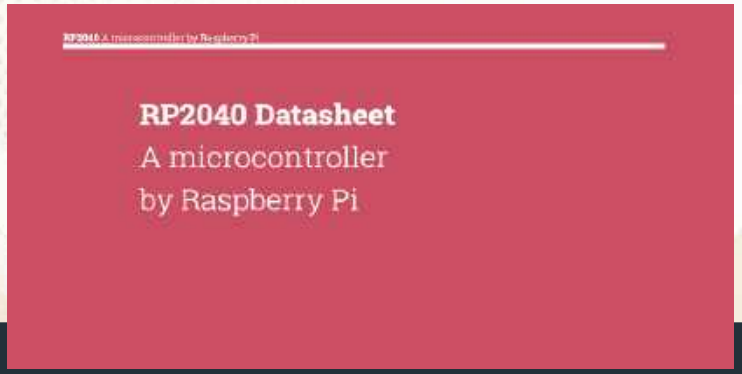
Ref 2



Ref 3



Ref 4



Ref 5